

Faculty of Engineering and Built Environment

Department of Electrical, Electronics and Computer Engineering

Course: Bachelor's in computer engineering

Group 24 members:

Tyokwana Alupheli	240912675
Zwane Banele	231187548
Majija Lilitha	240098137
Mbuzwana Lindokuhle	230173179
Dube Mluleki Peacemaker	250245949
Gumede Njabulo	251151018
Mamso Sanele John	231309325
Mvunelo Siyanda	251248003

Topic: Text Analyzer Tool

Content page

1. Introduction
    - 1.1. Project definition
    - 1.2. Project Objectives
    - 1.3. Scopes and Limitations
2. System designs and architecture
    - 2.1. Architectural overview
    - 2.2. UML class diagram
    - 2.3. Class description and responsibilities
    - 2.4. Data flow
3. Implementation details
    - 3.1 Core features and functionality
    - 3.2 Advanced Java features
4. Testing and evaluation
    - 4.1 Test cases
    - 4.2 Results and performance analysis
    - 4.3 System limitations
5. Graduate attribution reflection
    - 5.1 GA2: Application of scientific and engineering knowledge
    - 5.2 GA7: Sustainability and impact of engineering activity
6. Conclusion
7. References

1. Introduction

1.1 Problem definition

Text analyser is a digital tool to measure, evaluate and count written content to help one understand the structure, tone and vocabulary. Its need in this information driven world of large texts created and consumed simultaneously daily is very visible. Analysing large text manually for insights such as spelling error, readability and word frequency can be time consuming and prone to human error. Students, researchers and writers need a reliable way of obtaining summaries and statistics from written material and without an automated tool it is hard to identify the potential problems in the text. The goal of this project is to address this problem by developing a tool that automates the process of analysing text and provides a summary of the important findings.

1.2. Project objective

The Text analyser tool is a modular Java desktop application for performing multifaceted text analysis in an offline environment without the need for an external network. It will provide a correct way to analyse the text files or any direct input for the users. This application has two entry points: one for console users, and one for graphical desktop users.

1.3 Scope and limitations

Scope: The result of the project is a working Java desktop application for analysis of written text on his/her own computer. It accepts typed text,.txt files and in the graphical version,.pdf documents. It tells you how many characters, words and sentences are in each text, how many times each word occurs, possible spelling mistakes and an overall sentiment. Reports can be saved as a text file, and events and errors can be written to a log file. The tool is not available.

Limitations: The tool only analyses English text. The spell check and sentiment analysis are based on word lists, so they are not as accurate as commercial tools. Scanned PDFs, other file types (like Word documents), and very large files are not fully supported. Limitations are discussed in more detail in Section 4.3.

2. System designs and architecture

2.1 Architectural overview

The Text Analyser Tool is a Java desktop application layered so that the code that talks to the user is separate from the code that does the analysis.

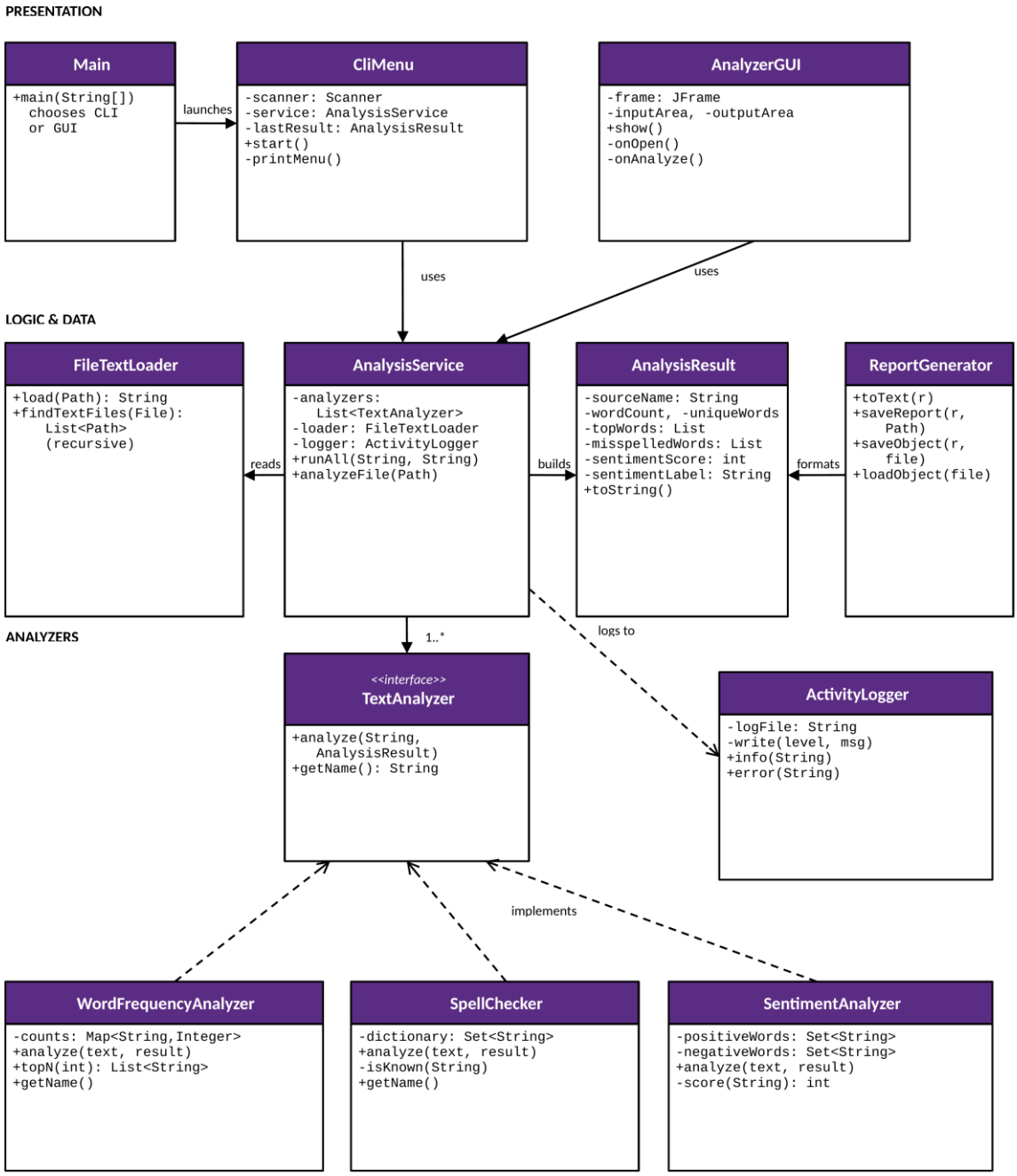
 This separation allows

the console menu and graphical window to both use the same analysis code and neither have any analysis logic of their own.

The system has four parts:

- **User interface:** Program starts Main. CliMenu displays a text menu in the console. AnalyzerGUI displays a Swing window where the user can write text or open a .txt or .pdf file. These classes simply gather input and output results.
- **Coordination:** AnalysisService forms the core of the program; it takes in the text, determines the basic counts, and then sends the text to each of the analyzers in turn. FileTextLoader assists it by reading files and folders from the disk.
- **Analysis modules:** Every analysis is carried out by a separate class which implements the TextAnalyzer interface, such as WordFrequencyAnalyzer, SpellChecker and SentimentAnalyzer. The service does not need to know the details of what each of these classes does; it only uses the analyze method, which means that a new analyzer can be added without any changes being made to the existing code. TextUtils carries out the common word-cleaning step.
- **Results, reports and logging:** The various analyzers place their results into a single shared AnalysisResult object, after which ReportGenerator converts this into a readable report and ActivityLogger logs the events and errors into activity.log.

2.2 UML class diagram



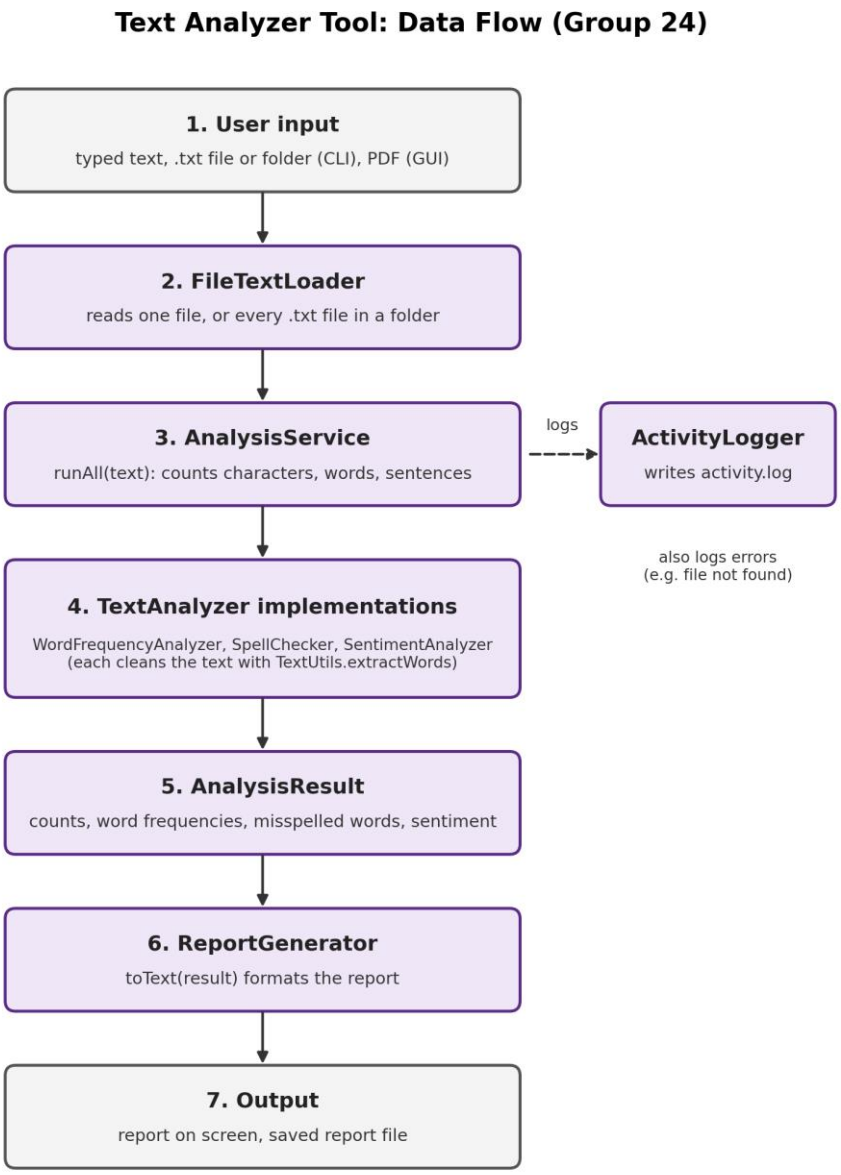
2.3 Class description and responsibilities

Class	Responsibility	Key methods
Main	Entry point . Starts application and displays the graphical interface.	main(String[])
CliMenu	Console menu with five options: analyze typed text, analyze file or folder, show last result, save report, exit.	`start()`, `printMenu()`, `handleChoice(int)`

AnalyzerGUI	Swing window with input area, report area and buttons: Open File, Analyze Text, Clear, Save Report. It imports.txt and .pdf files.	show()
AnalysisService	Controls the analysis. Counts the number of characters, words and sentences, and logs the event.	`runAll(String)`, `analyzeFile(Path)`
FileTextLoader	Reads text files line by line. Reads all .txt files in a folder and any subfolders.	`load(Path)`, `loadDirectory(Path)`
`TextAnalyzer` (interface)	All analysers must comply with this contract.	`analyze(String, AnalysisResult)`, `getName()`
WordFrequencyAnalyzer	Counts the number of occurrences of each word. Sorts the words by frequency, most frequent first.	`analyze(...)`, `topN(int)`, `getName()`
SpellChecker	Loads a dictionary file into a set and logs words not in the dictionary.	`analyze(...)`, `getName()`
SentimentAnalyzer	It scores the text as positive words minus negative words and labels it as Positive, Negative or Neutral.	`analyze(...)`, `getName()`
AnalysisResult	Holds the results of one analysis: character counts, word count, sentence count, word frequencies, misspelt words and sentiment. It is serializable	getters and setters, `toString()`
ReportGenerator	Formats a result as a text report and writes it to a file, or writes or loads the result object.	`toText(...)`, `saveReport(...)`, `saveObject(...)`, `loadObject(...)`
ActivityLogger	Append event and error messages to activity.log with time stamp.	`info(String)`, `error(String)`
TextUtils	The analyzers use a helper class. It divides text into	extractWords(String)

	words, eliminates punctuation, and changes the font to lowercase.	

2.4 Data flow



3. Implementation details

3.1 Core features and functionality

- **Input and output:** In the console version, the program reads entire folders,.txt files, and user-typed text. Additionally, the GUI imports.txt and.pdf files. The results can be saved to a text file and displayed as a report on the screen.
- **Processing logic:** AnalysisService counts words, sentences, and characters (with and without spaces). Word frequencies, potential spelling errors (checked against a dictionary file), and a sentiment score are then added by three analyzers.

- **Data structures:** The word counts are stored in a HashMap, and their sorted order is maintained by a LinkedHashMap. ArrayLists store words and misspelled words, while hash sets store the dictionary and lists of positive and negative words.
- **Algorithms and recursion:** FileTextLoader uses Files to search a folder and all of its subfolders.stroll. Words with the same count are arranged alphabetically, and the word list is sorted by count from highest to lowest.
- **Logging and reporting:** Every analysis and error is noted in activity.log along with a timestamp. For every analysis, ReportGenerator generates a formatted report.
- **Two interfaces:** A five-option console menu (CliMenu) and a Swing window (AnalyzerGUI) with buttons for Open File, Analyze Text, Clear, and Save Report.

3.2 Advanced Java features

- **Interface and polymorphism:** AnalysisService calls analyze on each item in a List<TextAnalyzer> without identifying the analyzer. It is possible to add a new analyzer without altering the current code.
- **Generics and collections:** Type-safe storage is offered through Map, List, and Set.
- **Recursion and lambda expressions:** Lambdas are used to filter the files and sort the word counts while Files.walk traverses the folder tree.
- **File handling:** Files can be read and written by BufferedReader, FileReader, FileWriter, and PrintWriter. Even in the event of an error, try-with-resources automatically closes every file.
- **Object serialization:** ReportGenerator has saveObject and loadObject methods to save a result and to load it later. AnalysisResult implements Serialisable.
- **Multithreading:** In order to prevent the window from freezing on large documents, the GUI uses SwingWorker to load files and perform the analysis in a background thread.
- **Regular expressions:** They identify numbers, separate text into words and sentences, and eliminate punctuation.
- **Exception handling:** File errors are reported to the user and recorded in the log.
- **External library:** Can be extracted from PDF files using Apache PDFBox.

Complexity of the main operations (N = characters in the text, W = number of words, U = unique words, K = misspelled words, F = files in a folder):

Operation	Complexity	Note
Character, word and sentence counts	O(N)	One pass through the text
Word frequency counting	O(W) on average	`HashMap` lookups
Sorting the frequencies	O(U log U)	Comparator sort
Dictionary lookup	O(1) on average	`HashSet`
Recording a misspelled word once	O(K) per word	Uses `ArrayList.contains`, so a set would be faster

Sentiment scoring	O(W)	`HashSet` lookups
Folder search	O(F)	Visits every entry once

4. Testing and evaluation

4.1 Test cases

ID	Test	Input	Expected result	Actual result (from the screenshots)	Pass / Fail
T1	Normal text, no spelling error	`Hello world`	words, 1 sentence, Neutral (0), no misspellings, `hello: 1, world: 1`	12 characters (10 without spaces), 2 words, 1 sentence, Neutral (0), no misspellings, `hello: 1, world: 1`	Pass
T2	Spell checker	`Hellow world`	2 words, `hellow` flagged as misspelled	13 characters (11 without spaces), 2 words, 1 sentence, Neutral (0), misspelled: `[hellow]`	Pass
T3	`.txt` file, no errors	A text file: “Java is a powerful object-oriented programming language. Unit testing and clean architecture ensure software success. Excellent work”	17 words, 3 sentences, Positive (+3) from “clean”, “success” and “excellent”, no misspellings	133 characters (116 without spaces), 17 words, 3 sentences, Positive (+3), no misspellings	Pass
T4	`.txt` file with a spelling error	The same file with “work” typed as `worki`	The same counts, with `worki` flagged	134 characters (117 without spaces), 17 words, 3 sentences, Positive (+3), misspelled: `[worki]`	Pass

4.2 Results and performance analysis

Each of the four tests that were run produced the desired results. The tool generated the sentiment result, listed the word frequencies, and accurately counted characters, words, and sentences. The misspelled words "hellow" and "worki" were flagged by the spell checker against the dictionary in T2 and T4, but nothing was flagged in T1 and T3. Three positive words—"clean," "success," and "excellent"—produced the sentiment Positive (+3) in T3 and T4. The identical handling of typed text and an uploaded.txt file demonstrates the interoperability of the GUI, AnalysisService, and analyzers.

Observations

• For instance, "Hello world" has 11 characters, but the "Characters (with spaces)" figure is one higher than the visible text. The count includes the trailing space or line break that was probably present in the text box. Without spaces, the count is precise.
• The frequency list most likely displays "object" and "oriented" separately, but a hyphenated word like "object-oriented" counts as one word because the word count splits on spaces.

Performance: There was no discernible delay in the completion of all four runs. We are unable to make any claims regarding large inputs because we did not measure timing with large files.

Restrictions on the testing: One text file and typed text are covered in the tests. The PDF and report-saving features, as well as edge cases (empty input, a missing file, protected or extremely large files), have not yet been tested, and the results do not demonstrate their functionality.

4.3 System limitations

- **English only:** The text cleaning keeps only the letters a to z and digits, so accented characters are removed (for example “café” becomes “caf”), and text in non-Latin scripts is lost.
- **Apostrophes and hyphens split words:** "Well-known" is read as "well" and "known," and "don't" is read as "don" and "t." As a result, the number of entries in the frequency list may differ from the word count, which splits on spaces.

- **The spell checker only compares against a word list:** There are roughly 50,000 words in the dictionary file, but some typing errors are acceptable because it also contains single letters and non-words. Names and technical terms may be flagged, but it is unable to recommend corrections or identify a correct word used incorrectly (like "their" and "there"). The spell check is bypassed and only a console warning is displayed if the dictionary file cannot be located.
- **Sentiment analysis is simple:** It subtracts one count from the other using brief lists of thirteen positive and thirteen negative words. It does not comprehend sarcasm, negation (“not good” scores as positive), or word strength.
- **Sentence counting is approximate:** Abbreviations like "Dr." and numbers like "3.14" can add more sentences because a sentence is counted whenever a full stop, question mark, or exclamation mark appears.
- **PDF support is limited:** You can only read PDFs that are not encrypted and have selectable text. PDFs with columns may appear out of order, and scanned images require OCR, which is not included.
- **A folder is analyzed as one combined text:** A folder is examined as a single, integrated text. Only.txt files within a folder are included, and the results are not displayed per file.
- **Whole files are read into memory:** Complete files are loaded into memory. Large files may run out of memory or be slow.
- **Reports are plain text:** The activity log continues to grow because nothing clears or rotates it, and there are no charts or other formats.

5. Graduate attribution reflection

5.1 GA2: Application of scientific and engineering knowledge

This project required us to apply software engineering and computer science knowledge to solve a real problem: analyzing text quickly and consistently.

- **Object-oriented design:** we split the system into classes with one job each and used an interface (`TextAnalyzer`) so that the analyzers can be changed or added without affecting the rest of the program.
- **Data structures:** we chose `HashMap` and `HashSet` for fast lookups and used sorting to rank words by frequency.
- **Algorithms and complexity:** We took into account each operation's cost, such as O(1) dictionary lookups, O(N) counting, and O(U log U) sorting.
- **Recursion and file handling:** The program can safely read and write files using try-with-resources, search folders and their subfolders, and save results using serialization.
- **Concurrency:** To prevent large files from freezing the window, the graphical user interface uses a background thread.

- **Testing and error handling:** We handled errors by displaying a message and documenting them in the log, and we tested both standard and edge cases (empty input, missing files, protected PDFs).

5.2 GA7: Sustainability and impact of engineering activity

- **Efficient use of resources:** The program doesn't use network or cloud resources because it runs solely on the user's computer and doesn't require a server or internet connection. Because hash-based data structures reduce processing time, each analysis requires less processor time and energy.
- **Reduced manual effort:** Decreased manual effort: manual text analysis is laborious and prone to mistakes. Students, researchers, and writers can save time by automating it.
- **Maintainable and reusable design:** The software lasts longer and doesn't need to be rewritten from scratch because of its modular structure and comments, which make it simpler to modify, expand, and reuse.
- **Open and low-cost tools:** Because the project makes use of free tools (Java, NetBeans, and Apache PDFBox), there are no license fees associated with its use or improvement
- **Limits to be honest about:** These advantages result from design decisions rather than measured outcomes because we did not measure energy use.

6. Conclusion

The issue of rapidly and reliably analyzing written text is resolved by the Text Analyzer Tool. While events and errors are documented in a log file, the user who types text or uploads a.txt file in a graphical window receives a report that includes character, word, and sentence counts, word frequencies, potential spelling errors, and an overall sentiment. The project's goal of producing a functional offline Java application utilizing regular expressions, collections, object-oriented design, and file handling was achieved.

The group can work on multiple classes simultaneously thanks to the program's layered design and TextAnalyzer interface, which also makes it simple to expand. Although edge cases like empty input and very large files still need to be checked, testing with typed text and text files produced the expected results. The tool's shortcomings are listed in Section 4.3, and enhancements might include a bigger dictionary, better negation and apostrophe handling, and support for additional file types. We learned from working as a team how important it is to decide on shared classes early on, like AnalysisResult, so that everyone's code works together.

7. References

1. Apache Software Foundation (n.d.) *Apache NetBeans IDE*. Available at: https://netbeans.apache.org/ (Accessed: [date]).
2. Apache Software Foundation (n.d.) *Apache PDFBox*. Available at: https://pdfbox.apache.org/ (Accessed: [date]).
3. Bloch, J. (2018) *Effective Java*. 3rd edn. Boston: Addison-Wesley.
4. Cape Peninsula University of Technology (2026) *SDN260S Software Design II: Project Brief and lecture notes*. Cape Town: CPUT.
5. Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th edn. Cambridge, MA: MIT Press.
6. Fowler, M. (2004) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Boston: Addison-Wesley.
7. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
8. Oracle (n.d.) *Collections (The Java Tutorials)*. Available at: https://docs.oracle.com/javase/tutorial/collections/ (Accessed: [date]).
9. Oracle (n.d.) *Concurrency in Swing (The Java Tutorials)*. Available at: https://docs.oracle.com/javase/tutorial/uiswing/concurrency/ (Accessed: [date]).
10. Oracle (n.d.) *Creating a GUI With Swing (The Java Tutorials)*. Available at: https://docs.oracle.com/javase/tutorial/uiswing/ (Accessed: [date]).
11. Oracle (n.d.) *Essential Java Classes (The Java Tutorials)*. Available at: https://docs.oracle.com/javase/tutorial/essential/ (Accessed: [date]).
12. Oracle (n.d.) *Java Platform, Standard Edition Documentation*. Available at: https://docs.oracle.com/en/java/ (Accessed: [date]).
13. Oracle (n.d.) *Java SE API Specification*. Available at: https://docs.oracle.com/en/java/javase/ (Accessed: [date]).
14. Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.